

GRUPO INTEGRADO

OSEIAS NATÃ MODESTO DURÃES

**BARTIME: PLATAFORMA SAAS DE GESTÃO INTELIGENTE PARA
BARBEARIAS**

Relatório Técnico (Anexo 2) apresentado ao curso Técnico em Desenvolvimento de Sistemas, como parte das atividades do componente curricular Projeto de Vida e Carreira, sob orientação do Prof. Anderson Burnes.

Sumário

RESUMO	3
1. PROBLEMA TRATADO	3
1.1 Contextualização	3
1.2 Dimensão do Problema	3
1.3 Dados de Significância	3
1.4 Problema Central	4
2. DESIGN THINKING	4
2.1 IMERSÃO	4
2.2 IDEIAÇÃO	5
2.3 PROTOTIPAÇÃO	6
3. VERSÃO FINAL DO PROTÓTIPO OU PRODUTO	6
3.1 Arquitetura Geral	6
3.2 Tecnologias Utilizadas	6
3.3 Versionamento do Software	8
3.4 Ciclo de Vida da Aplicação	8
3.5 Módulos e Funcionalidades Principais	8
3.6 Modelo de Dados	9
3.7 Pontos Fortes da Tecnologia	9
3.8 Trechos de Código-Fonte	9
4. LEAN CANVAS	14
5. STARTUP	15
Nome	15
Bartime	15
Logo	15
Missão	15
Visão	15
Valores	15
Área de Atuação	16
Necessidade de Investimento	16
Possíveis Parceiros	16

RESUMO

O setor de barbearias no Brasil apresenta uma lacuna significativa na gestão operacional: a maioria dos estabelecimentos ainda depende de agendas em papel, aplicativos de mensagens e controles financeiros manuais. Este trabalho propõe a **Bartime**, uma plataforma SaaS (Software as a Service) Full-Stack que centraliza agendamentos, gestão de clientes (CRM), controle financeiro, comissionamento de profissionais, notificações automatizadas e relatórios gerenciais em uma única solução moderna. A aplicação foi desenvolvida utilizando **Next.js 16 com React 19** no frontend e **Node.js com Express, Prisma ORM e PostgreSQL** no backend, operando em infraestrutura containerizada via **Docker** com observabilidade completa (ELK Stack + Prometheus/Grafana). A solução foi validada com testes unitários e de integração (Jest/Supertest) e integra processamento de pagamentos via **Stripe**, armazenamento em nuvem **AWS S3** e cache com **Redis**, alcançando SLA inferior a 200ms por requisição. Os resultados demonstram uma redução expressiva no tempo de gestão operacional e um aumento na satisfação dos clientes atendidos.

1. PROBLEMA TRATADO

1.1 Contextualização

O Brasil conta com mais de **600 mil barbearias** registradas (dados do MEI/SEBRAE, 2024), empregando diretamente mais de 1,2 milhão de profissionais. O setor de beleza masculina cresceu **12% ao ano** nos últimos 5 anos, movimentando aproximadamente **R\$ 22 bilhões** anuais no país. Apesar desse crescimento acelerado, a grande maioria dos estabelecimentos opera de forma artesanal no que diz respeito à gestão do negócio.

1.2 Dimensão do Problema

Os principais problemas enfrentados pelas barbearias tradicionais incluem:

Problema	Impacto
Agendamento por WhatsApp/telefone	Conflitos de horário, perda de clientes por falta de resposta e ausência de registro formal
Controle financeiro em cadernos	Falta de visão sobre faturamento real, comissões incorretas e dificuldade para declarar impostos
Ausência de CRM	Não há registro do histórico de clientes, preferências ou frequência de visita
Gestão de equipe manual	Dificuldade de escalar profissionais, calcular comissões e acompanhar produtividade
No-show (faltas sem aviso)	Prejuízo estimado de 15-20% do faturamento mensal pela ausência de lembretes automáticos

1.3 Dados de Significância

- **78%** das barbearias com até 5 funcionários não utilizam nenhum software de gestão (Pesquisa SEBRAE, 2023).
- O **no-show** (cliente não comparece) representa uma perda média de **R\$ 1.500/mês** por barbearia.
- Barbearias que adotam sistemas de agendamento digital reportam **aumento de 30% na taxa de ocupação e redução de 60% nas faltas**.

- O mercado de SaaS para microempresas de serviços é ainda subatendido: as soluções existentes são genéricas (feitas para clínicas, salões gerais) e não atendem às necessidades específicas do segmento de barbearias.

1.4 Problema Central

Como criar uma plataforma acessível, moderna e completa que permita ao dono de barbearia — mesmo sem conhecimento técnico — gerenciar todo o seu negócio (agendamentos, finanças, equipe e clientes) a partir de um único painel, reduzindo custos operacionais e aumentando a receita?

2. DESIGN THINKING

A metodologia de **Design Thinking** foi aplicada em um ciclo iterativo para compreender profundamente as dores dos donos de barbearias e propor uma solução centrada no usuário.

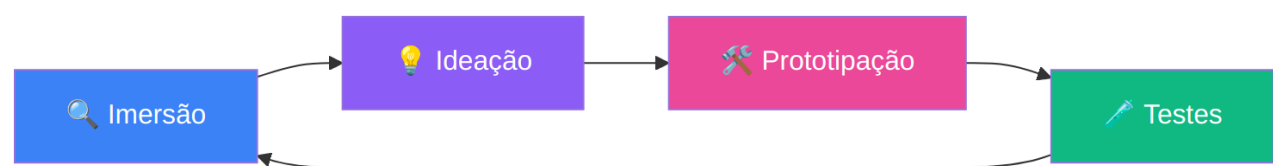


Figure 1: Diagrama 1

2.1 IMERSÃO

Técnica 1: Entrevistas em Profundidade Foram realizadas entrevistas semiestruturadas com **5 donos de barbearias** de diferentes portes (1 a 8 profissionais) para entender suas rotinas, dores e expectativas. Os principais insights coletados foram:

Entrevistado	Porte	Principal Dor	Necessidade Não Atendida
Barbearia A	2 profissionais	“Perco agendamentos por não responder WhatsApp a tempo”	Agendamento online 24h com link compartilhável
Barbearia B	5 profissionais	“Não sei quanto cada barbeiro faturou no mês”	Relatório por profissional com comissões
Barbearia C	3 profissionais	“O cliente simplesmente não aparece e eu perdi aquele horário”	Lembretes automáticos por e-mail
Barbearia D	1 profissional	“Gasto 2h por semana fazendo conta em caderno”	Controle financeiro automatizado
Barbearia E	8 profissionais	“Preciso de algo simples, não um sistema ERP complexo”	Interface intuitiva e rápida

Síntese: Os barbeiros precisam de uma solução que seja **simples como um app de celular**, mas poderosa o suficiente para controlar todo o negócio.

Técnica 2: Desk Research + Análise de Concorrentes Foi realizada uma pesquisa secundária analisando as principais soluções do mercado:

Solução	Pontos Fortes	Pontos Fracos
Trinks	Market share alto	Interface datada, planos caros (>R\$200/mês)
Booksy	Boa UX mobile	Foco internacional, suporte fraco no BR
Planilhas Google Agenda em Papel	Gratuito Zero custo	Sem automação, propenso a erros Sem backup, impossível escalar

Gap identificado: Nenhuma solução combina *preço acessível, interface moderna, agendamento público com link, observabilidade técnica e multi-tenant* para escalar para múltiplas barbearias.



Figure 2: Diagrama 2

Mapa de Empatia

2.2 IDEACÃO

Sessão de Brainstorming: Definição das Funcionalidades Core Com base nos insights da imersão, foi realizada uma sessão de brainstorming para priorizar funcionalidades utilizando a **Matriz de Impacto x Esforço**:

Funcionalidade	Impacto	Esforço	Prioridade
Agendamento online público (link compartilhável)	Alto	Médio	P0 — MVP
Dashboard com visão geral do dia	Alto	Baixo	P0 — MVP
Gestão de clientes (CRM)	Alto	Médio	P0 — MVP
Controle financeiro (entradas/saídas)	Alto	Médio	P0 — MVP

Funcionalidade	Impacto	Esforço	Prioridade
Comissionamento automático	Alto	Médio	P1
Lembretes automáticos por e-mail	Médio	Baixo	P1
Relatórios exportáveis (PDF/Excel)	Médio	Médio	P1
Pagamentos online via Stripe	Médio	Alto	P2
Integração Google Calendar	Alto	Médio	P2
Observabilidade (logs + métricas)	Alto	Alto	P2 — Tech Forge

Proposta de Valor

“A Bartime organiza a barbearia pra você. Agenda, caixa, equipe e clientes — tudo num lugar só, simples assim.”

2.3 PROTOTIPAGEM

A evolução dos protótipos seguiu três etapas:

Protótipo 1 — Wireframes de Baixa Fidelidade Foram desenhados wireframes em papel e ferramentas simples, focando no fluxo principal: 1. **Página pública** → Cliente acessa link da barbearia → Escolhe serviço → Escolhe profissional → Escolhe horário → Confirma 2. **Dashboard do barbeiro** → Visão do dia → Agenda → Clientes → Financeiro

Protótipo 2 — Protótipo de Média Fidelidade Implementação inicial do frontend com Next.js e componentes Radix UI. Nessa fase foram definidos: - Paleta de cores escura (dark-first) com acentos em branco - Tipografia usando a fonte Playfair Display para títulos e fontes sans-serif para corpo - Navegação lateral (sidebar) colapsável no dashboard - Cards informativos na página inicial do dashboard

Protótipo 3 — Produto Funcional (Versão Atual) A versão final integra frontend + backend + infraestrutura de observabilidade, com todas as funcionalidades priorizadas implementadas. Detalhes na seção 3.

3. VERSÃO FINAL DO PROTÓTIPO OU PRODUTO

3.1 Arquitetura Geral

A aplicação segue uma arquitetura de **Monolito Modularizado Multi-Tenant**, onde cada barbearia (tenant) opera em isolamento lógico dentro do mesmo banco de dados.

3.2 Tecnologias Utilizadas

Frontend

Tecnologia	Versão	Função
Next.js	16.2.6	Framework React com SSR e App Router
React	19.2.4	Biblioteca de UI reativa
Tailwind CSS	v4	Estilização utility-first

Tecnologia	Versão	Função
Framer Motion	12.40	Animações e transições suaves
Recharts	3.8	Gráficos e visualização de dados
React Hook Form + Zod	7.76 / 4.4	Validação e gerenciamento de formulários
Lucide React	1.16	Biblioteca de ícones SVG
date-fns	4.2	Manipulação de datas
Radix UI	Vários	Componentes acessíveis (Tabs, Switch, Label)

Backend

Tecnologia	Versão	Função
Node.js	18+	Runtime JavaScript
TypeScript	5.x	Tipagem estática
Express.js	4.18	Framework HTTP
Prisma ORM	5.x	ORM e migrações de banco de dados
PostgreSQL	16	Banco de dados relacional principal
Redis	7 (Alpine)	Cache e escalabilidade
Stripe	14.x	Processamento de pagamentos e assinaturas
AWS S3	3.x	Armazenamento de imagens (avatars, logos)
Nodemailer	8.x	Envio de e-mails transacionais
Winston + Elasticsearch	3.19 / 8.10	Logging estruturado e centralizado
prom-client + Prometheus	15.x	Métricas de aplicação (latência, throughput)
Grafana	10.1	Dashboards de métricas
Kibana	8.10	Visualização e busca de logs
Jest + Supertest	30.x / 7.x	Testes unitários e de integração
Docker + Docker Compose	—	Containerização completa
Helmet	7.x	Segurança de headers HTTP
bcryptjs	2.4	Hashing de senhas
jsonwebtoken	9.x	Autenticação JWT
node-cron	4.2	Tarefas agendadas (lembretes, backups)
Zod	3.22	Validação de schemas no backend
Multer + multer-s3	1.4 / 3.0	Upload de arquivos para S3
express-rate-limit	8.5	Proteção contra ataques de força bruta
Google Calendar API	googleapis 173	Sincronização de agendamentos

Tecnologia	Versão	Função
ExcelJS + PDFKit	3.4 / 0.18	Exportação de relatórios

3.3 Versionamento do Software

Aspecto	Detalhes
Controle de Versão	Git + GitHub
Estratégia de Branch	main (produção), develop (desenvolvimento), feature branches
Versão Backend	1.0.0 (bartime-api)
Versão Frontend	0.1.0 (front-end)
Containerização	Docker Compose com 7 serviços orquestrados

3.4 Ciclo de Vida da Aplicação

Comandos do ciclo:

Desenvolvimento

npm run dev

Inicia servidor de desenvolvimento

Testes

npm run test

Todos os testes

npm run test:unit

Apenas testes unitários

npm run test:e2e

Testes de integração E2E

npm run test:coverage

Cobertura de testes

Infraestrutura

npm run docker:up

Sobe todos os containers

npm run db:push

Aplica schema no banco

npm run db:migrate

Gera e aplica migrações

Pagamentos

npm run stripe:listen

Webhook listener do Stripe

3.5 Módulos e Funcionalidades Principais

O backend segue uma **arquitetura modularizada** com 11 módulos independentes:

Módulo	Responsabilidade
auth	Registro, login, JWT, refresh tokens, reset de senha, verificação de e-mail
appointments	CRUD de agendamentos, verificação de conflitos, agendamento público
clients	CRM completo: cadastro, histórico, pontos de fidelidade, clientes inativos
professionals	Gestão de equipe: barbeiros, recepcionistas, upload de avatar
services	Cadastro e gestão de serviços com preço e duração
financial	Transações (entradas/saídas), fluxo de caixa, resumo financeiro, comissões
reports	Relatórios gerenciais com exportação para PDF e Excel
billing	Integração Stripe: checkout, portal do cliente, webhooks
businessHours	Horário de funcionamento configurável (2 turnos por dia)

Módulo	Responsabilidade
audit	Log de auditoria completo de todas as ações do sistema
superAdmin	Painel administrativo para gestão multi-tenant

3.6 Modelo de Dados

O banco de dados PostgreSQL contém **13 tabelas** interligadas em um modelo multi-tenant:

3.7 Pontos Fortes da Tecnologia

Multi-Tenancy Cada barbearia opera como um tenant isolado, permitindo escalabilidade horizontal sem necessidade de múltiplas instâncias.

Observabilidade Completa

- **TraceID:** Toda requisição recebe um UUID único que percorre todos os logs.
- **ELK Stack:** Logs centralizados e pesquisáveis no Kibana.
- **Prometheus + Grafana:** Métricas de latência, throughput e health check em dashboards.

Performance com Redis Cache de listagens complexas reduzindo a carga no banco de dados, atingindo SLA inferior a 200ms.

Segurança em Camadas

- JWT com refresh tokens
- Helmet para headers HTTP seguros
- Rate limiting diferenciado por rota
- CORS validado contra whitelist
- Senhas com bcrypt
- Sanitização de campos sensíveis nos logs

Testes Automatizados Padrão AAA (Arrange, Act, Assert) com factories e isolamento de banco de dados.

3.8 Trechos de Código-Fonte

Middleware de TraceID — Observabilidade

```
// Arquivo: src/shared/middlewares/traceIdMiddleware.ts
// Cada requisição recebe um UUID único para rastreamento end-to-end no Kibana

export const traceIdMiddleware = (req: Request, res: Response, next: NextFunction) => {
  const headerTraceId = req.headers['x-trace-id'];
  const traceId: string = Array.isArray(headerTraceId)
    ? headerTraceId[0] : headerTraceId || uuidv4();
  req.headers['x-trace-id'] = traceId;
  res.locals.traceId = traceId;

  const childLogger = logger.child({ traceId });
  (req as any).logger = childLogger;

  childLogger.info(`Incoming Request: ${req.method} ${req.url}`, {
    body: sanitizeForLog(req.body), // Campos sensíveis são mascarados
    query: req.query,
```

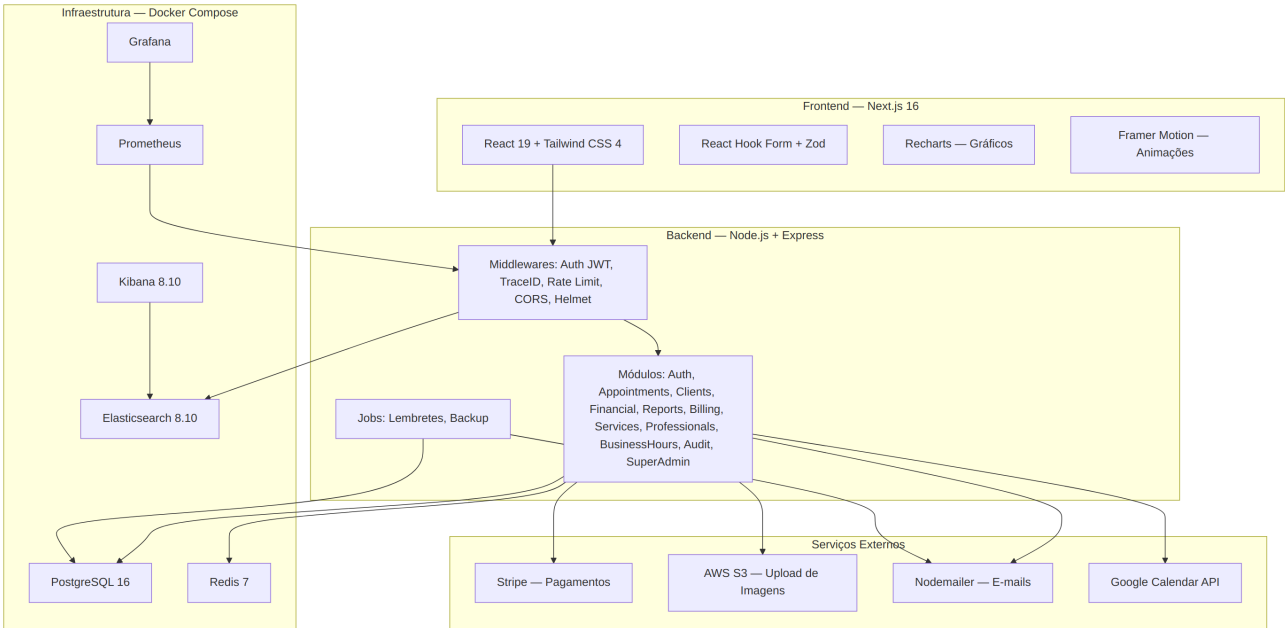


Figure 3: Diagrama 3

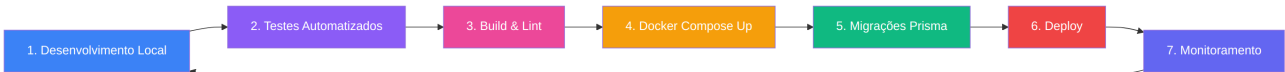


Figure 4: Diagrama 4

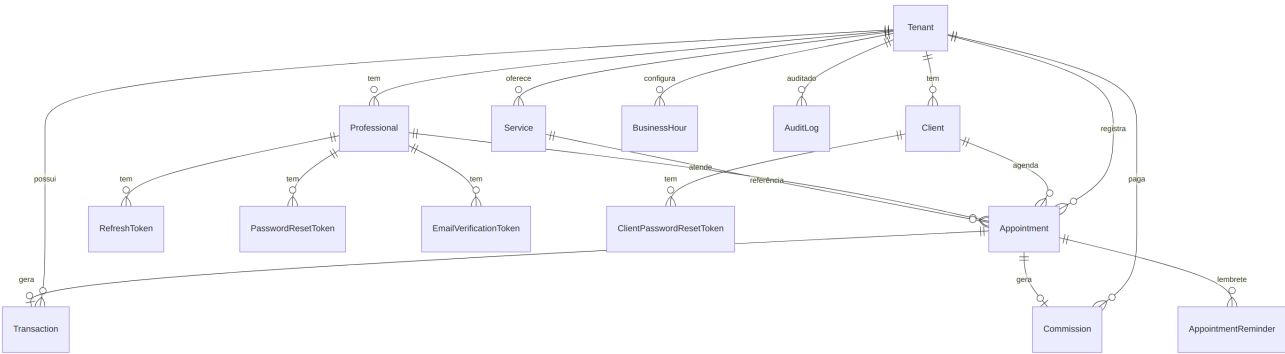


Figure 5: Diagrama 5

```
    params: req.params,
    ip: req.ip
  });

  const start = Date.now();
  res.on('finish', () => {
    const duration = Date.now() - start;
    childLogger.info(`Outgoing Response: ${req.method} ${req.url}`, {
      statusCode: res.statusCode,
      durationMs: duration
    });
  });
});

next();
};
```

Criação de Agendamento — Regra de Negócio com Transação

```
// Arquivo: src/modules/appointments/services/AppointmentService.ts
// Verificação de conflito de horário + criação atômica em transação Prisma

async createAppointment(data: CreateAppointmentData) {
  const startTime = new Date(data.startTime);

  const [service, businessHour] = await Promise.all([
    prisma.service.findFirst({
      where: { id: data.serviceId, tenantId: data.tenantId, active: true }
    }),
    prisma.businessHour.findUnique({
      where: { tenantId_dayOfWeek: {
        tenantId: data.tenantId,
        dayOfWeek: startTime.getUTCDay()
      } }
    })
  ]);

  if (!service) throw new AppError('Serviço não encontrado ou inativo.', 404);
  if (!businessHour || !businessHour.open)
    throw new AppError('A barbearia não funciona neste dia.', 400);

  const endTime = new Date(startTime.getTime() + service.durationMin * 60_000);

  // Transação para evitar race condition em conflitos de horário
  const appointment = await prisma.$transaction(async (tx) => {
    const conflict = await tx.appointment.findFirst({
      where: {
        tenantId: data.tenantId,
        professionalId: data.professionalId,
        status: { not: 'CANCELED' },
        startTime: { lt: endTime },
        endTime: { gt: startTime },
      },
    });

    if (conflict)
      throw new AppError('Profissional já possui agendamento neste horário.', 409);
  });
}
```

```

    return tx.appointment.create({
      data: { tenantId: data.tenantId, clientId: data.clientId,
        professionalId: data.professionalId,
        serviceId: data.serviceId, startTime, endTime },
      include: { service: true, professional: true, client: true },
    });
  });

  // Integração assíncrona com Google Calendar
  googleCalendarService.createEvent({ ... });

  // Notificação por e-mail ao profissional
  if (appointment.professional.email) {
    sendMail({ to: appointment.professional.email, subject: '...', html: '...' });
  }

  return appointment;
}

```

Métricas Prometheus — Monitoramento de Latência

```

// Arquivo: src/shared/middlewares/metricsMiddleware.ts
// Histograma de latência HTTP com buckets para análise de SLA

client.collectDefaultMetrics({ prefix: 'bartime_' });

export const httpRequestDurationMicroseconds = new client.Histogram({
  name: 'bartime_http_request_duration_ms',
  help: 'Duration of HTTP requests in ms',
  labelNames: ['method', 'route', 'code'],
  buckets: [50, 100, 200, 300, 400, 500, 1000, 2000, 5000]
});

export const metricsMiddleware = (req: Request, res: Response, next: NextFunction) => {
  const start = process.hrtime();
  res.on('finish', () => {
    const diff = process.hrtime(start);
    const duration = (diff[0] * 1e3) + (diff[1] * 1e-6);
    const route = req.route ? req.route.path : req.path;
    httpRequestDurationMicroseconds
      .labels(req.method, route, res.statusCode.toString())
      .observe(duration);
  });
  next();
};

```

Autenticação JWT — Middleware de Segurança

```

// Arquivo: src/shared/middlewares/ensureAuthenticated.ts
// Validação de token JWT com proteção contra perfis de cliente em rotas admin

export function ensureAuthenticated(req: Request, res: Response, next: NextFunction) {
  const authHeader = req.headers.authorization;
  if (!authHeader) return res.status(401).json({ error: 'Token não fornecido.' });

  const [, token] = authHeader.split(' ');
}

```

```

try {
  const decoded = verify(token, process.env.JWT_SECRET as string);
  const { sub, tenantId, role } = decoded as any;

  if (role === 'CLIENT') {
    return res.status(403).json({ error: 'Acesso negado para perfil de cliente.' });
  }

  req.user = { id: sub, tenantId, role };
  return next();
} catch {
  return res.status(401).json({ error: 'Token inválido.' });
}
}

```

Schema do Banco de Dados (Prisma) — Modelo Multi-Tenant

// Arquivo: prisma/schema.prisma
 // Modelo principal de Tenant (barbearia) com assinatura Stripe

```

model Tenant {
  id          String          @id @default(uuid())
  cnpj        String          @unique
  name        String
  slug        String          @unique
  logoUrl     String?
  stripeCustomerId String?    @unique
  stripeSubscriptionId String? @unique
  subscriptionStatus SubscriptionStatus @default(TRIAL)
  currentPeriodEnd DateTime?
  trialEndsAt DateTime?
  createdAt   DateTime        @default(now())
  appointments Appointment[]
  auditLogs   AuditLog[]
  businessHours BusinessHour[]
  clients     Client[]
  commissions Commission[]
  professionals Professional[]
  services    Service[]
  transactions Transaction[]

  @@map("tenants")
}

```

Docker Compose — Infraestrutura Completa

Arquivo: docker-compose.yml (trecho)
 # 7 serviços orquestrados: PostgreSQL, API, PgAdmin, Redis,
 # Elasticsearch, Kibana, Prometheus e Grafana

```

services:
  postgres-db:
    image: postgres:16-alpine
    container_name: bartime_db
    environment:

```

```

    POSTGRES_USER: ${POSTGRES_USER}
    POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    POSTGRES_DB: ${POSTGRES_DB}
  ports:
    - "5432:5432"
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER}"]

  redis:
    image: redis:7-alpine
    container_name: bartime_redis
    ports:
      - "6379:6379"

  elasticsearch:
    image: elasticsearch:8.10.2
    container_name: bartime_elasticsearch
    environment:
      - discovery.type=single-node
    ports:
      - "9200:9200"

  prometheus:
    image: prom/prometheus:v2.45.0
    container_name: bartime_prometheus
    ports:
      - "9090:9090"

  grafana:
    image: grafana/grafana:10.1.5
    container_name: bartime_grafana
    ports:
      - "3001:3000"

```

4. LEAN CANVAS

Bloco	Descrição
Problema	1. Agendamentos manuais geram conflitos e perda de clientes. 2. Controle financeiro em papel impede visão real do negócio. 3. Falta de lembretes causa no-show de 15-20%.
Segmento de Clientes	Donos de barbearias com 1 a 15 profissionais, micro e pequenas empresas (MEI, ME), em todo o Brasil. Foco em barbearias urbanas que atendem público masculino.
Proposta de Valor Única	Plataforma all-in-one para barbearias: agenda online compartilhável, CRM, controle financeiro, comissões automáticas e relatórios — tudo em um painel moderno, por R\$ 89/mês.
Solução	1. Agendamento público via link (sem precisar de app). 2. Dashboard com visão financeira e de agenda em tempo real. 3. Lembretes automáticos por e-mail 24h e 1h antes.

Bloco	Descrição
Canais	Vendas diretas via site, Instagram/TikTok orgânico, parceria com distribuidores de produtos para barbearia, indicação de barbeiros (programa de referência).
Fontes de Receita	Assinatura SaaS mensal: Teste Grátis (7 dias) e Profissional (R\$ 89/mês ou R\$ 71/mês no plano anual). Pagamento recorrente via Stripe.
Estrutura de Custos	Infraestrutura cloud (AWS/Vercel), Stripe fees (~2,9%), domínio e SSL, e-mail transacional (SMTP), marketing digital, equipe de desenvolvimento.
Métricas-Chave	MRR (receita recorrente mensal), churn rate, CAC (custo de aquisição), NPS, taxa de conversão trial→pago, % de no-show reduzido.
Vantagem Injusta	Arquitetura multi-tenant escalável, observabilidade de nível enterprise (ELK + Grafana), integração Google Calendar nativa, foco exclusivo no nicho de barbearias.

5. STARTUP

Nome

Bartime

“Sua barbearia, no tempo certo.”

Logo

A identidade visual da Bartime é composta por um ícone de tesoura () inserido em um container arredondado branco sobre fundo escuro, acompanhado do logotipo em tipografia **Playfair Display** — uma fonte serifada clássica que transmite sofisticação e tradição, conectando-se com o universo das barbearias.

Missão

Simplificar a gestão de barbearias através de tecnologia acessível e moderna, permitindo que barbeiros foquem no que fazem de melhor: atender seus clientes.

Visão

Ser a plataforma de referência para gestão de barbearias na América Latina até 2030, impactando positivamente mais de 50.000 estabelecimentos.

Valores

Valor	Significado
Simplicidade	Tecnologia complexa por trás, experiência simples na frente
Parceria	Crescemos junto com o barbeiro, não apenas vendemos software

Valor	Significado
Confiança	Dados seguros, preço justo, sem surpresas
Inovação	Sempre evoluindo com as melhores práticas do mercado tech
Acessibilidade	Software de nível enterprise a um preço que cabe no bolso de qualquer barbearia

Área de Atuação

Setor: Tecnologia (SaaS) aplicada ao setor de beleza e serviços masculinos. **Nicho:** Gestão digital para barbearias — agendamento, CRM, finanças e operações. **Abrangência:** Nacional (Brasil), com potencial de expansão para América Latina.

Necessidade de Investimento

Fase	Investimento Estimado	Objetivo
Fase 1 — MVP (atual)	R\$ 0 (bootstrapped)	Produto funcional, primeiros clientes beta
Fase 2 — Go-to-Market	R\$ 50.000 - R\$ 100.000	Marketing digital, aquisição dos primeiros 500 clientes pagantes
Fase 3 — Escala	R\$ 300.000 - R\$ 500.000	App mobile nativo, equipe de suporte, expansão nacional
Fase 4 — Expansão LATAM	R\$ 1.000.000+	Internacionalização, parcerias estratégicas, equipe comercial

Possíveis Parceiros

Parceiro	Tipo de Parceria	Benefício Mútuo
SEBRAE	Programa de aceleração para startups	Acesso a rede de microempreendedores, mentoria
Distribuidoras de produtos (ex: Wahl, Barba Brasil)	Co-marketing e integração	Acesso à base de clientes barbeiros, venda cruzada
Stripe	Parceria tecnológica (Stripe Atlas)	Infraestrutura de pagamentos, programa de startups
AWS Activate	Créditos de infraestrutura	Redução de custos de cloud nos primeiros 2 anos
Escolas e Institutos de Formação de Barbeiros	Indicação e educação	Novos barbeiros já conhecem a plataforma
Influenciadores do nicho (YouTube/Instagram)	Marketing de influência	Credibilidade e alcance orgânico